

🔪 ДОМЕН 3: C++ — ПОЛНАЯ ДЕТАЛИЗАЦИЯ (ОТ НУЛЯ ДО ЭКСПЕРТА)

СТРУКТУРА КАЖДОЙ ТЕМЫ

Для каждой темы я даю:

- **Что нужно знать** — теория для понимания
 - **Вопросы на собеседовании** (Junior → Senior → Staff)
 - **Практика** — конкретные задания для закрепления
-

РАЗДЕЛ 3.1: БАЗОВЫЙ СИНТАКСИС И ООП (JUNIOR LEVEL)

3.1.1. Основы языка (Junior Level)

№ 3.1.1.1 — Типы данных

- **Что нужно знать:** int, float, double, char, bool, void, размеры на разных платформах
- **Вопросы на собеседовании:** "Сколько байт занимает int на 64-битной системе?"
- **Практика:** Написать программу, выводящую sizeof всех базовых типов

№ 3.1.1.2 — Переменные

- **Что нужно знать:** объявление, инициализация, область видимости (scope), const (константы)
- **Вопросы на собеседовании:** "В чем разница между const int* и int* const?"
- **Практика:** Написать примеры всех комбинаций const с указателями и объяснить

№ 3.1.1.3 — Указатели

- **Что нужно знать:** * (разыменование), & (взятие адреса), арифметика указателей, nullptr
- **Вопросы на собеседовании:** "Что такое арифметика указателей? Приведи пример"
- **Практика:** Написать программу, демонстрирующую арифметику указателей для массива

№ 3.1.1.4 — Ссылки (references)

- **Что нужно знать:** lvalue references, отличие от указателей (не могут быть nullptr, не переназначаются)
- **Вопросы на собеседовании:** "В чем разница между указателем и ссылкой?"
- **Практика:** Написать функцию, принимающую параметр по ссылке и меняющую его

№ 3.1.1.5 — Условные операторы

- **Что нужно знать:** if, else, switch, тернарный оператор ? :
- **Вопросы на собеседовании:** "Как написать цепочку условий в C++?"
- **Практика:** Написать программу, определяющую оценку (A/B/C/D/F) по баллу

№ 3.1.1.6 — Циклы

- **Что нужно знать:** for, while, do-while, break, continue
- **Вопросы на собеседовании:** "Как выйти из вложенного цикла?"
- **Практика:** Написать программу, проверяющую, является ли число простым

№ 3.1.1.7 — Функции

- **Что нужно знать:** объявление, определение, передача по значению/ссылке, перегрузка (overload)
- **Вопросы на собеседовании:** "Что такое перегрузка функций?"
- **Практика:** Написать функцию max для двух int, двух double, трех int

№ 3.1.1.8 — Массивы

- **Что нужно знать:** статические (int arr[10]), многомерные, передача в функцию (размер теряется)
- **Вопросы на собеседовании:** "Как передать массив в функцию в C++?"
- **Практика:** Написать функцию print_array, принимающую массив и размер

№ 3.1.1.9 — Перечисления

- **Что нужно знать:** enum (plain), enum class (C++11) — строго типизированные
- **Вопросы на собеседовании:** "В чем преимущество enum class перед enum?"

- **Практика:** Написать enum class Color и использовать в switch

№ 3.1.1.10 — Пространства имен

- **Что нужно знать:** namespace, using (потенциальные конфликты)
 - **Вопросы на собеседовании:** "Зачем нужны namespace?"
 - **Практика:** Создать свой namespace my_math с функциями add, sub, mul
-

3.1.2. Классы и ООП (Junior → Middle Level)

№ 3.1.2.1 — Классы и структуры

- **Что нужно знать:** разница (все public по умолчанию в struct), методы, поля
- **Вопросы на собеседовании:** "В чем разница между class и struct в C++?"
- **Практика:** Создать класс Point с полями x, y и методом distance_to

№ 3.1.2.2 — Конструкторы

- **Что нужно знать:** по умолчанию, с параметрами, копирования, перемещения (C++11), делегирующие (C++11)
- **Вопросы на собеседовании:** "Что такое конструктор копирования? Когда он вызывается?"
- **Практика:** Создать класс String с конструкторами, деструктором

№ 3.1.2.3 — Деструкторы

- **Что нужно знать:** ~ClassName(), освобождение ресурсов, virtual destructor
- **Вопросы на собеседовании:** "Почему деструктор должен быть виртуальным?"
- **Практика:** Создать иерархию Base/Derived и показать, что происходит без virtual destructor

№ 3.1.2.4 — Модификаторы доступа

- **Что нужно знать:** public, protected, private (для полей и методов)
- **Вопросы на собеседовании:** "Что такое инкапсуляция?"

- **Практика:** Создать класс с private полями и public геттерами/сеттерами

№ 3.1.2.5 — Наследование

- **Что нужно знать:** public (основное), protected, private наследование
- **Вопросы на собеседовании:** "В чем разница между public и private наследованием?"
- **Практика:** Создать класс Shape → Circle/Rectangle, добавить виртуальные методы

№ 3.1.2.6 — Виртуальные функции

- **Что нужно знать:** virtual, override (C++11), final (C++11), vtable
- **Вопросы на собеседовании:** "Как работает виртуальная функция под капотом?"
- **Практика:** Создать иерархию с виртуальным методом draw(), дизассемблировать и показать vtable

№ 3.1.2.7 — Абстрактные классы

- **Что нужно знать:** pure virtual (virtual void f() = 0;)
- **Вопросы на собеседовании:** "Что такое pure virtual функция?"
- **Практика:** Создать абстрактный класс Animal с pure virtual make_sound()

№ 3.1.2.8 — Статические члены

- **Что нужно знать:** static поля (одна копия на класс), static методы
- **Вопросы на собеседовании:** "Что такое статический метод? Можно ли вызвать его без объекта?"
- **Практика:** Создать класс Counter со статическим полем count и методом get_count()

№ 3.1.2.9 — Дружественные функции

- **Что нужно знать:** friend (нарушает инкапсуляцию)
- **Вопросы на собеседовании:** "Что такое friend функция? Когда она полезна?"
- **Практика:** Создать класс Complex и friend функцию для оператора +

№ 3.1.2.10 — Операторы

- **Что нужно знать:** перегрузка +, -, ==, <<, >>, [], ()
 - **Вопросы на собеседовании:** "Как перегрузить оператор << для своего класса?"
 - **Практика:** Перегрузить операторы для класса Complex (сложение, вычитание, умножение)
-

3.1.3. STL (Standard Template Library) — Junior Level

№ 3.1.3.1 — Контейнеры

- **Что нужно знать:** std::vector, std::string, std::array (фиксированный размер)
- **Вопросы на собеседовании:** "Когда использовать std::array вместо C-массива?"
- **Практика:** Написать программу, использующую std::vector для хранения чисел

№ 3.1.3.2 — Итераторы

- **Что нужно знать:** begin(), end(), rbegin(), rend(), категории итераторов (input, forward, bidirectional, random access)
- **Вопросы на собеседовании:** "Что такое итераторы? Какие категории бывают?"
- **Практика:** Написать цикл с итератором для std::vector и std::list

№ 3.1.3.3 — Алгоритмы

- **Что нужно знать:** std::sort, std::find, std::for_each, std::accumulate
- **Вопросы на собеседовании:** "Как отсортировать вектор и найти элемент?"
- **Практика:** Написать программу, сортирующую вектор строк и находящую подстроку

№ 3.1.3.4 — Ассоциативные контейнеры

- **Что нужно знать:** std::map (красно-черное дерево), std::unordered_map (хеш-таблица)
- **Вопросы на собеседовании:** "В чем разница между std::map и std::unordered_map?"

- **Практика:** Написать программу, подсчитывающую частоту слов в тексте с `std::map`

№ 3.1.3.5 — Контейнеры множеств

- **Что нужно знать:** `std::set`, `std::unordered_set`
- **Вопросы на собеседовании:** "Когда использовать `std::set` вместо `std::unordered_set`?"
- **Практика:** Написать программу для поиска уникальных элементов в массиве

№ 3.1.3.6 — Адаптеры контейнеров

- **Что нужно знать:** `std::stack`, `std::queue`, `std::priority_queue`
- **Вопросы на собеседовании:** "Как использовать `std::priority_queue`?"
- **Практика:** Реализовать алгоритм Дейкстры с `std::priority_queue`

3.1.4. Практические задания (Junior Level)

№ 3.1.4.1

- **Задание:** Реализовать класс `String` с конструкторами, деструктором, операторами
- **Что проверяет:** Понимание управления памятью и ООП

№ 3.1.4.2

- **Задание:** Реализовать иерархию фигур (`Shape` → `Circle`/`Rectangle`/`Triangle`) с виртуальным методом `area()`
- **Что проверяет:** Понимание наследования и полиморфизма

№ 3.1.4.3

- **Задание:** Написать программу, читающую CSV файл и выводящую статистику (среднее, минимум, максимум)
- **Что проверяет:** Работа с STL и файлами

№ 3.1.4.4

- **Задание:** Реализовать класс `Complex` с перегрузкой арифметических операторов
 - **Что проверяет:** Перегрузка операторов
-

РАЗДЕЛ 3.2: СОВРЕМЕННЫЙ C++ (C++11/14/17/20) — MIDDLE LEVEL

3.2.1. C++11/14 — Ключевые нововведения (Middle Level)

№ 3.2.1.1 — auto и decltype

- **Что нужно знать:** автоматическое выведение типа, decltype для выведения типа выражения
- **Вопросы на собеседовании:** "Когда использовать auto? Какие есть подводные камни?"
- **Практика:** Написать программу, использующую auto для итераторов, лямбд, сложных типов

№ 3.2.1.2 — Range-based for

- **Что нужно знать:** for (auto& x : container)
- **Вопросы на собеседовании:** "Как работает range-based for?"
- **Практика:** Написать цикл по std::vector и std::map с range-based for

№ 3.2.1.3 — nullptr

- **Что нужно знать:** замена NULL (0) для указателей
- **Вопросы на собеседовании:** "Почему лучше использовать nullptr вместо NULL?"
- **Практика:** Написать программу с перегрузкой функции для int и void* и показать разницу

№ 3.2.1.4 — enum class

- **Что нужно знать:** строго типизированные перечисления (без неявного приведения к int)
- **Вопросы на собеседовании:** "В чем преимущество enum class перед enum?"
- **Практика:** Написать enum class Color и использовать в switch без каста

№ 3.2.1.5 — Лямбды

- **Что нужно знать:** синтаксис [capture](parameters) -> return_type { body }, захват по значению/ссылке
- **Вопросы на собеседовании:** "Как работает захват в лямбдах? Что такое mutable лямбда?"
- **Практика:** Написать лямбду, сортирующую вектор по второму полю структуры

№ 3.2.1.6 — Обобщенные лямбды (C++14)

- **Что нужно знать:** auto в параметрах
- **Вопросы на собеседовании:** "В чем отличие C++11 лямбды от C++14?"
- **Практика:** Написать лямбду, работающую с любым типом контейнера

№ 3.2.1.7 — Move-семантика

- **Что нужно знать:** rvalue references (&&), std::move, std::forward
- **Вопросы на собеседовании:** "Что такое move-семантика? В чем отличие от копирования?"
- **Практика:** Реализовать класс Buffer с move-конструктором и move-присваиванием

№ 3.2.1.8 — Правило пяти (Rule of Five)

- **Что нужно знать:** деструктор, конструктор копирования, конструктор перемещения, оператор присваивания, оператор перемещения
- **Вопросы на собеседовании:** "Что такое правило пяти?"
- **Практика:** Написать класс с динамической памятью и реализовать все 5 методов

№ 3.2.1.9 — Умные указатели

- **Что нужно знать:** std::unique_ptr (эксклюзивное владение), std::shared_ptr (разделяемое владение), std::weak_ptr (для предотвращения циклических ссылок)
- **Вопросы на собеседовании:** "В чем разница между unique_ptr и shared_ptr?"
- **Практика:** Реализовать класс Resource и показать работу с unique_ptr и shared_ptr

№ 3.2.1.10 — std::make_unique / std::make_shared

- **Что нужно знать:** безопасное создание умных указателей
- **Вопросы на собеседовании:** "Почему лучше использовать make_unique вместо new?"
- **Практика:** Создать unique_ptr через make_unique, shared_ptr через make_shared

№ 3.2.1.11 — Строковые литералы

- **Что нужно знать:** сырые строки (R"..."), суффиксы для типов (auto s = "hello"s)
 - **Вопросы на собеседовании:** "Что такое сырые строки в C++?"
 - **Практика:** Написать программу с JSON в сырой строке
-

3.2.2. C++17 — Must have (Middle → Senior Level)

№ 3.2.2.1 — std::optional

- **Что нужно знать:** обертка для значения, которое может отсутствовать
- **Вопросы на собеседовании:** "Когда использовать std::optional вместо nullptr или исключений?"
- **Практика:** Написать функцию parse_int, возвращающую std::optional<int>

№ 3.2.2.2 — std::variant

- **Что нужно знать:** type-safe union (хранит один из набора типов)
- **Вопросы на собеседовании:** "В чем отличие std::variant от union?"
- **Практика:** Написать программу с std::variant<int, double, std::string> и std::visit

№ 3.2.2.3 — std::any

- **Что нужно знать:** тип, который может содержать любое значение (type erasure)
- **Вопросы на собеседовании:** "Что такое std::any и когда его использовать?"
- **Практика:** Создать вектор std::vector<std::any> и сохранить в нем разные типы

№ 3.2.2.4 — std::filesystem

- **Что нужно знать:** работа с файловой системой (пути, файлы, директории)
- **Вопросы на собеседовании:** "Как получить список файлов в директории с помощью C++17?"
- **Практика:** Написать программу, обходящую директорию и выводящую иерархию файлов

№ 3.2.2.5 — std::string_view

- **Что нужно знать:** невладеющее представление строки (быстрый доступ без копирования)
- **Вопросы на собеседовании:** "В чем преимущество `std::string_view` перед `const std::string&`?"
- **Практика:** Написать функцию `starts_with`, принимающую `std::string_view`

№ 3.2.2.6 — `if constexpr`

- **Что нужно знать:** compile-time ветвление в шаблонах
- **Вопросы на собеседовании:** "Когда использовать `if constexpr`?"
- **Практика:** Написать шаблонную функцию `print_type`, выводящую разный текст для разных типов

№ 3.2.2.7 — Fold Expressions

- **Что нужно знать:** свертка для variadic templates (C++17)
- **Вопросы на собеседовании:** "Что такое fold expressions?"
- **Практика:** Написать функцию `sum`, складывающую произвольное количество аргументов

№ 3.2.2.8 — Inline variables

- **Что нужно знать:** объявление переменных в заголовочных файлах без нарушения ODR
- **Вопросы на собеседовании:** "Как объявить глобальную переменную в заголовочном файле без проблем с ODR?"
- **Практика:** Создать `inline` переменную в header и использовать в нескольких `.cpp` файлах

№ 3.2.2.9 — `std::parallel` алгоритмы

- **Что нужно знать:** параллельные версии STL алгоритмов (`std::execution::par`)
- **Вопросы на собеседовании:** "Как ускорить `std::sort` с помощью параллелизма?"
- **Практика:** Написать программу, сортирующую 10 млн чисел с `std::execution::par`

3.2.3. C++20 — Будущее (Senior Level)

№ 3.2.3.1 — Concepts

- **Что нужно знать:** ограничения для шаблонов (замена SFINAE), requires
- **Вопросы на собеседовании:** "Что такое Concepts и зачем они нужны?"
- **Практика:** Написать концепт Sortable и функцию, требующую его

№ 3.2.3.2 — Coroutines

- **Что нужно знать:** co_await, co_yield, co_return, std::generator (C++23)
- **Вопросы на собеседовании:** "Как работают корутины в C++20?"
- **Практика:** Написать генератор чисел Фибоначчи с co_yield

№ 3.2.3.3 — Ranges Library

- **Что нужно знать:** std::ranges, views, actions (lazy evaluation)
- **Вопросы на собеседовании:** "Что такое Ranges и в чем их преимущество?"
- **Практика:** Написать цепочку filter и transform с Ranges

№ 3.2.3.4 — std::span

- **Что нужно знать:** невладеющее представление непрерывной последовательности
- **Вопросы на собеседовании:** "В чем отличие std::span от std::vector?"
- **Практика:** Написать функцию, принимающую std::span<int> и обрабатывающую его

№ 3.2.3.5 — std::jthread

- **Что нужно знать:** поток с автоматическим join() при уничтожении
- **Вопросы на собеседовании:** "Чем std::jthread отличается от std::thread?"
- **Практика:** Написать программу с std::jthread и std::stop_token для остановки

№ 3.2.3.6 — std::stop_token

- **Что нужно знать:** механизм остановки потоков
- **Вопросы на собеседовании:** "Как безопасно остановить поток в C++20?"
- **Практика:** Написать поток, который проверяет std::stop_token в цикле

№ 3.2.3.7 — `std::atomic` улучшения

- **Что нужно знать:** `std::atomic<bool>`, `std::atomic<shared_ptr>`
- **Вопросы на собеседовании:** "Как атомарно обновить `shared_ptr`?"
- **Практика:** Написать программу с `std::atomic<std::shared_ptr>`

№ 3.2.3.8 — `std::format`

- **Что нужно знать:** форматирование строк (как Python f-strings)
- **Вопросы на собеседовании:** "Как отформатировать строку в C++20?"
- **Практика:** Написать программу, выводящую информацию в форматированном виде

№ 3.2.3.9 — `std::bit_cast`

- **Что нужно знать:** безопасный `reinterpret_cast` для примитивных типов
- **Вопросы на собеседовании:** "В чем отличие `std::bit_cast` от `reinterpret_cast`?"
- **Практика:** Написать программу, конвертирующую `float` в `uint32_t` через `bit_cast`

№ 3.2.3.10 — `std::barrier` и `std::latch`

- **Что нужно знать:** точки синхронизации для потоков
- **Вопросы на собеседовании:** "В чем разница между `barrier` и `latch`?"
- **Практика:** Написать программу с `std::latch` для ожидания запуска всех потоков

3.2.4. Практические задания (Middle → Senior Level)

№ 3.2.4.1

- **Задание:** Реализовать класс `Buffer` с move-семантикой и правилом пяти
- **Что проверяет:** Понимание управления ресурсами

№ 3.2.4.2

- **Задание:** Написать программу, использующую `std::optional`, `std::variant`, `std::any`
- **Что проверяет:** Знание современных типов

№ 3.2.4.3

- **Задание:** Реализовать `std::vector` с использованием итераторов и аллокаторов
- **Что проверяет:** Глубокое понимание контейнеров

№ 3.2.4.4

- **Задание:** Написать корутину-генератор чисел Фибоначчи (C++20)
- **Что проверяет:** Понимание корутин

№ 3.2.4.5

- **Задание:** Реализовать концепт `Printable` и функцию `print` для любого типа, поддерживающего `operator<<`
- **Что проверяет:** Понимание Concepts

РАЗДЕЛ 3.3: ШАБЛОНЫ И МЕТАПРОГРАММИРОВАНИЕ (SENIOR → STAFF)

3.3.1. Шаблоны (Senior Level)

№ 3.3.1.1 — Шаблоны функций

- **Что нужно знать:** `template<typename T> T max(T a, T b)`
- **Вопросы на собеседовании:** "Как работает инстанцирование шаблонов?"
- **Практика:** Написать шаблонную функцию `swap` для произвольного типа

№ 3.3.1.2 — Шаблоны классов

- **Что нужно знать:** `template<typename T> class Stack { ... }`
- **Вопросы на собеседовании:** "Что такое специализация шаблона?"
- **Практика:** Написать шаблон `Container<T>` с методами `add` и `get`

№ 3.3.1.3 — Специализация шаблонов

- **Что нужно знать:** полная (`template<> class Stack<int>`) и частичная (`template<typename T> class Stack<T*>`)
- **Вопросы на собеседовании:** "В чем разница между полной и частичной специализацией?"
- **Практика:** Специализировать `Container<bool>` для хранения битов

№ 3.3.1.4 — Параметры шаблонов

- **Что нужно знать:** типы, значения (non-type template parameters), шаблоны (template template parameters)
- **Вопросы на собеседовании:** "Как передать значение как параметр шаблона?"
- **Практика:** Написать шаблон `Array<T, N>` с фиксированным размером `N`

№ 3.3.1.5 — Variadic templates (C++11)

- **Что нужно знать:** шаблоны с переменным числом аргументов
- **Вопросы на собеседовании:** "Как реализовать `printf`-подобную функцию с variadic templates?"
- **Практика:** Написать функцию `print`, выводящую произвольное количество аргументов

№ 3.3.1.6 — SFINAE (Substitution Failure Is Not An Error)

- **Что нужно знать:** `std::enable_if`, определение, работает ли функция для типа
- **Вопросы на собеседовании:** "Что такое SFINAE? Как использовать `enable_if`?"
- **Практика:** Написать шаблон, который работает только для целочисленных типов

№ 3.3.1.7 — Type Traits

- **Что нужно знать:** `std::is_integral`, `std::is_class`, `std::is_pointer` и другие
- **Вопросы на собеседовании:** "Что такое type traits?"
- **Практика:** Написать функцию `assert_type`, проверяющую тип во время компиляции

№ 3.3.1.8 — CRTP (Curiously Recurring Template Pattern)

- **Что нужно знать:** `class Derived : public Base<Derived>`
- **Вопросы на собеседовании:** "Что такое CRTP и зачем он нужен?"
- **Практика:** Реализовать статический полиморфизм через CRTP

№ 3.3.1.9 — Policy-based design

- **Что нужно знать:** стратегии (policy classes) как параметры шаблона
- **Вопросы на собеседовании:** "Что такое policy-based design?"

- **Практика:** Написать класс SmartPtr с политикой управления памятью (refcounted / intrusive)

№ 3.3.1.10 — Концепты (C++20)

- **Что нужно знать:** requires, concept — замена SFINAE
- **Вопросы на собеседовании:** "В чем преимущество Concepts перед SFINAE?"
- **Практика:** Написать концепт Number и функцию, требующую его

№ 3.3.1.11 — Fold expressions (C++17)

- **Что нужно знать:** (... + args), (args + ...), (args, ...)
- **Вопросы на собеседовании:** "Как работает fold expression?"
- **Практика:** Написать функцию sum, складывающую произвольное количество аргументов

3.3.2. Метапрограммирование (Staff Level)

№ 3.3.2.1 — Компиляторные вычисления

- **Что нужно знать:** `template<int N> struct Factorial { static const int value = N * Factorial<N-1>::value; };`
- **Вопросы на собеседовании:** "Как вычислить факториал во время компиляции?"
- **Практика:** Реализовать компиляторный вариант factorial и pow

№ 3.3.2.2 — Type lists

- **Что нужно знать:** работа с списками типов во время компиляции (boost::mpl / mp11)
- **Вопросы на собеседовании:** "Что такое type list?"
- **Практика:** Реализовать список типов и операции push_back, pop_front

№ 3.3.2.3 — Compile-time detection

- **Что нужно знать:** проверка существования метода у типа (SFINAE)
- **Вопросы на собеседовании:** "Как проверить, есть ли у класса метод foo()?"
- **Практика:** Написать trait has_foo для проверки существования метода

№ 3.3.2.4 — decltype и declval

- **Что нужно знать:** работа с типами без создания объектов
- **Вопросы на собеседовании:** "Что такое declval и когда его использовать?"
- **Практика:** Написать выражение SFINAE с decltype и declval

№ 3.3.2.5 — Tag dispatching

- **Что нужно знать:** использование пустых структур для выбора перегрузки
- **Вопросы на собеседовании:** "Что такое tag dispatching?"
- **Практика:** Написать функцию, переключающуюся между алгоритмами по типу итератора

№ 3.3.2.6 — Expression templates

- **Что нужно знать:** ленивые вычисления, оптимизация (например, в Eigen)
- **Вопросы на собеседовании:** "Что такое expression templates?"
- **Практика:** Реализовать выражение для сложения векторов без временных объектов

3.3.3. Практические задания (Senior → Staff Level)

№ 3.3.3.1

- **Задание:** Реализовать std::vector с аллокатором и итераторами
- **Что проверяет:** Понимание контейнеров и аллокаторов

№ 3.3.3.2

- **Задание:** Реализовать умный указатель shared_ptr с контролем ссылок
- **Что проверяет:** Понимание управления памятью

№ 3.3.3.3

- **Задание:** Реализовать std::tuple с помощью variadic templates
- **Что проверяет:** Глубокое понимание шаблонов

№ 3.3.3.4

- **Задание:** Реализовать std::function (type erasure)
- **Что проверяет:** Понимание type erasure

№ 3.3.3.5

- **Задание:** Реализовать compile-time парсер JSON (с помощью шаблонов)
 - **Что проверяет:** Экспертный уровень метапрограммирования
-

РАЗДЕЛ 3.4: МНОГОПОТОЧНОСТЬ И МОДЕЛЬ ПАМЯТИ (SENIOR → STAFF)

3.4.1. Потоки и синхронизация (Senior Level)

№ 3.4.1.1 — `std::thread`

- **Что нужно знать:** создание, `join()`, `detach()`, передача параметров
- **Вопросы на собеседовании:** "Как передать аргументы в поток?"
- **Практика:** Написать программу с 10 потоками, каждый выводит свой ID

№ 3.4.1.2 — `std::jthread` (C++20)

- **Что нужно знать:** автоматический `join`, `stop_token` для остановки
- **Вопросы на собеседовании:** "В чем преимущество `std::jthread` перед `std::thread`?"
- **Практика:** Написать поток, проверяющий `stop_token` в цикле

№ 3.4.1.3 — `std::mutex`

- **Что нужно знать:** `std::lock_guard`, `std::unique_lock` (RAII), `std::scoped_lock` (C++17)
- **Вопросы на собеседовании:** "Что такое RAII в контексте мьютексов?"
- **Практика:** Написать программу с `std::lock_guard` для защиты общего ресурса

№ 3.4.1.4 — `std::shared_mutex` (C++17)

- **Что нужно знать:** read-write lock
- **Вопросы на собеседовании:** "Когда использовать `shared_mutex`?"
- **Практика:** Реализовать кеш с `shared_mutex` для многопоточного доступа

№ 3.4.1.5 — `std::condition_variable`

- **Что нужно знать:** `wait()`, `notify_one()`, `notify_all()`

- **Вопросы на собеседовании:** "Как работает producer-consumer с condition_variable?"
- **Практика:** Реализовать producer-consumer с condition_variable

№ 3.4.1.6 — std::async

- **Что нужно знать:** асинхронный запуск, std::future, std::promise
 - **Вопросы на собеседовании:** "В чем разница между std::async и std::thread?"
 - **Практика:** Написать программу с std::async для параллельного вычисления
-

3.4.2. Атомарные операции и модель памяти (Staff Level)

№ 3.4.2.1 — std::atomic

- **Что нужно знать:** std::atomic<int>, std::atomic<bool>
- **Вопросы на собеседовании:** "Как реализовать счетчик без мьютекса?"
- **Практика:** Написать счетчик с std::atomic<int>

№ 3.4.2.2 — Memory ordering

- **Что нужно знать:** memory_order_relaxed, memory_order_acquire, memory_order_release, memory_order_acq_rel, memory_order_seq_cst
- **Вопросы на собеседовании:** "В чем разница между memory_order_relaxed и memory_order_seq_cst?"
- **Практика:** Написать программу, показывающую, как memory_order влияет на видимость

№ 3.4.2.3 — Happens-before relationship

- **Что нужно знать:** формальная модель памяти
- **Вопросы на собеседовании:** "Что такое happens-before?"
- **Практика:** Изучить формальную модель памяти C++ и написать примеры

№ 3.4.2.4 — Lock-free структуры

- **Что нужно знать:** Treiber stack, Michael-Scott queue
- **Вопросы на собеседовании:** "Как реализовать lock-free стек?"

- **Практика:** Реализовать lock-free стек (Treiber) с CAS

№ 3.4.2.5 — АВА проблема

- **Что нужно знать:** проблема в lock-free структурах и ее решение
- **Вопросы на собеседовании:** "Что такое АВА проблема?"
- **Практика:** Изучить решение АВА проблемы с помощью счетчика версий

№ 3.4.2.6 — `std::atomic<shared_ptr>` (C++20)

- **Что нужно знать:** атомарная работа с умными указателями
- **Вопросы на собеседовании:** "Как атомарно обновить `shared_ptr`?"
- **Практика:** Написать программу с `std::atomic<shared_ptr<Resource>>`

3.4.3. Практические задания (Senior → Staff Level)

№ 3.4.3.1

- **Задание:** Реализовать thread pool с очередью задач
- **Что проверяет:** Знание многопоточности

№ 3.4.3.2

- **Задание:** Реализовать lock-free стек (Treiber)
- **Что проверяет:** Понимание атомарных операций

№ 3.4.3.3

- **Задание:** Реализовать lock-free очередь (Michael-Scott)
- **Что проверяет:** Понимание lock-free структур

№ 3.4.3.4

- **Задание:** Написать программу с разными `memory_order` и показать различия
- **Что проверяет:** Понимание модели памяти

РАЗДЕЛ 3.5: БИБЛИОТЕКИ (QT, BOOST, ASIO, GTest) — SENIOR LEVEL

3.5.1. Qt (Кроссплатформенная GUI) — Senior Level

№ 3.5.1.1 — Signals & Slots

- **Что нужно знать:** механизм событий, connect(), emit, МОС (Meta-Object Compiler)
- **Вопросы на собеседовании:** "Как работают сигналы и слоты в Qt?"
- **Практика:** Написать приложение с кнопкой, отправляющей сигнал

№ 3.5.1.2 — Qt Widgets

- **Что нужно знать:** QWidget, QMainWindow, QPushButton, QLabel, QLineEdit
- **Вопросы на собеседовании:** "Как создать GUI на Qt?"
- **Практика:** Создать калькулятор с Qt Widgets

№ 3.5.1.3 — Qt Quick / QML

- **Что нужно знать:** декларативный UI, современный подход
- **Вопросы на собеседовании:** "Когда использовать QML вместо Widgets?"
- **Практика:** Написать небольшое приложение на QML

№ 3.5.1.4 — Qt Network

- **Что нужно знать:** QNetworkAccessManager (HTTP клиент), QTcpSocket, QUdpSocket
- **Вопросы на собеседовании:** "Как сделать HTTP запрос в Qt?"
- **Практика:** Написать программу, загружающую данные с REST API

№ 3.5.1.5 — Qt SQL

- **Что нужно знать:** QSqlDatabase, QSqlQuery, модели данных (QSqlTableModel)
- **Вопросы на собеседовании:** "Как работать с БД в Qt?"
- **Практика:** Написать приложение, отображающее таблицу из SQLite

№ 3.5.1.6 — Qt Multithreading

- **Что нужно знать:** QtConcurrent, QThread, QThreadPool
- **Вопросы на собеседовании:** "Как работать с потоками в Qt?"
- **Практика:** Написать программу с QtConcurrent для параллельной обработки данных

№ 3.5.1.7 — Qt Models

- **Что нужно знать:** QAbstractItemModel, QAbstractTableModel, работа с данными
 - **Вопросы на собеседовании:** "Как отобразить пользовательские данные в QListView?"
 - **Практика:** Реализовать модель для отображения списка пользователей
-

3.5.2. Boost (Библиотека) — Senior Level

№ 3.5.2.1 — Boost.Asio

- **Что нужно знать:** асинхронный ввод-вывод, TCP/UDP серверы, таймеры
- **Вопросы на собеседовании:** "Как работает Asio?"
- **Практика:** Написать TCP-сервер с Boost.Asio

№ 3.5.2.2 — Boost.Filesystem (теперь std::filesystem)

- **Что нужно знать:** работа с файловой системой
- **Вопросы на собеседовании:** "Как обойти директорию с Boost.Filesystem?"
- **Практика:** Написать программу для поиска всех .cpp файлов

№ 3.5.2.3 — Boost.Algorithm

- **Что нужно знать:** алгоритмы (boost::algorithm::string)
- **Вопросы на собеседовании:** "Какие алгоритмы есть в Boost?"
- **Практика:** Написать программу для обработки строк

№ 3.5.2.4 — Boost.JSON (C++11/14)

- **Что нужно знать:** работа с JSON, парсинг и сериализация
- **Вопросы на собеседовании:** "Как парсить JSON в C++?"
- **Практика:** Написать программу, читающую и создающую JSON

№ 3.5.2.5 — Boost.Beast

- **Что нужно знать:** HTTP/WebSocket библиотека на основе Asio
 - **Вопросы на собеседовании:** "Как написать HTTP сервер на C++?"
 - **Практика:** Написать HTTP сервер с Boost.Beast
-

3.5.3. Тестирование (GoogleTest / GoogleMock) — Senior Level

№ 3.5.3.1 — GoogleTest

- **Что нужно знать:** TEST(), ASSERT_*, EXPECT_*, Test Fixtures
- **Вопросы на собеседовании:** "Как писать unit-тесты на C++?"
- **Практика:** Написать тесты для класса String (конструкторы, операторы)

№ 3.5.3.2 — GoogleMock

- **Что нужно знать:** мокирование классов, EXPECT_CALL, MOCK_METHOD
- **Вопросы на собеседовании:** "Как мокировать зависимость в C++?"
- **Практика:** Написать мок для класса Database и протестировать UserService

№ 3.5.3.3 — Parametrized tests

- **Что нужно знать:** TEST_P, INSTANTIATE_TEST_SUITE_P
- **Вопросы на собеседовании:** "Как запустить один тест с разными параметрами?"
- **Практика:** Написать параметризованный тест для функции max

№ 3.5.3.4 — Coverage

- **Что нужно знать:** gcov, lcov, измерение покрытия кода
- **Вопросы на собеседовании:** "Как измерить покрытие кода тестами?"
- **Практика:** Настроить покрытие для своего проекта

3.5.4. Практические задания (Senior Level)

№ 3.5.4.1

- **Задание:** Написать GUI приложение на Qt с сетевой частью (загрузка данных с API)
- **Что проверяет:** Знание Qt и сети

№ 3.5.4.2

- **Задание:** Написать TCP-сервер с Boost.Asio, обрабатывающий множество клиентов

- **Что проверяет:** Знание асинхронного ввода-вывода

№ 3.5.4.3

- **Задание:** Написать unit-тесты для своего проекта с GoogleTest
- **Что проверяет:** Знание тестирования

№ 3.5.4.4

- **Задание:** Написать HTTP сервер на Boost.Beast
- **Что проверяет:** Знание сетевых протоколов

🔗 СВОДНЫЙ ЧЕК-ЛИСТ ДЛЯ ПОДГОТОВКИ К СОБЕСЕДОВАНИЮ

Junior Level (должен знать 100%)

Базовый синтаксис: типы, циклы, условия, функции

Указатели и ссылки

Классы: конструкторы, деструкторы, наследование, виртуальные функции

STL: vector, string, map, unordered_map, set

Итераторы и алгоритмы (sort, find)

Исключения (try/catch)

Middle Level (должен знать 100%)

Move-семантика (rvalue references, std::move, std::forward)

Умные указатели (unique_ptr, shared_ptr, weak_ptr)

Лямбды, auto, decltype

std::optional, std::variant, std::any

std::filesystem, std::string_view

Шаблоны (функции, классов, специализация)

std::thread, std::mutex, std::lock_guard, std::condition_variable

Senior Level (должен знать 100%)

Шаблоны: SFINAE, enable_if, type traits, CRTP, variadic templates

C++17: fold expressions, if constexpr, std::parallel

C++20: concepts, coroutines, ranges, std::jthread, std::barrier

Модель памяти: std::atomic, memory ordering (acquire/release/seq_cst)

Lock-free структуры

Библиотеки: Qt (Signals & Slots, Widgets), Boost.Asio, GoogleTest

Staff Level (должен знать 100%)

Метапрограммирование: compile-time вычисления, type lists

Expression templates

Type erasure (std::function)

Внутреннее устройство аллокаторов

Формальная модель памяти C++ (happens-before)

Разработка lock-free структур с нуля

🔧 ПРАКТИЧЕСКИЙ ПРОЕКТ ДЛЯ ЗАКРЕПЛЕНИЯ ДОМЕНА 3

Задание: Реализовать распределенную систему мониторинга (агент + сервер) на C++.

Шаги:

1. Агент собирает метрики (CPU, Memory, Disk) каждые 5 секунд
2. Отправляет данные на сервер по сети (gRPC или WebSocket)
3. Сервер принимает данные и сохраняет в очередь (lock-free queue)
4. Сервер обрабатывает данные (сохраняет в БД, отправляет уведомления)
5. Все компоненты должны быть покрыты unit-тестами
6. GUI для отображения метрик (Qt или Avalonia)

Проверка:

- Система должна выдерживать 1000+ агентов
- Не должно быть утечек памяти
- Время обработки данных < 100 мс